Strings in Java

Learning by Example

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-14

Outline

1. How to Use These Examples
2. Creating and Inspecting
3. Searching and Comparing
4. Transforming
5. Building Strings
6. Modern Java
7. Summary

How to Use These Examples

Running the examples

Every program on these slides is a file in `codeoutput/`; the output shown is what it really printed (Java 21).

macOS (Terminal)

`cd` into the folder of the slides, then

```
javac -d build codeoutput/Case.java
java -cp build Case
sh run_examples.sh          # all of them
```

Windows (Command Prompt)

`chcp 65001` first, so Turkish letters print correctly:

```
chcp 65001
javac -encoding UTF-8 -d build codeoutput\
      Case.java
java -cp build Case
```

`run_examples.sh` runs in *Git Bash*.

Each slide shows the code first; the output appears on the next click. Predict it before you click.

Creating and Inspecting

Five ways to get a String

codeoutput/Create.java **Java**

```java
public class Create {
    public static void main(String[] args) {
        String a = "Yeditepe";                  // literal
        String b = new String("Yeditepe");      // new object
        char[] letters = {'J', 'a', 'v', 'a'};
        String c = new String(letters);         // from chars
        String d = String.valueOf(2026);        // from a number
        String e = "" + 3.14;  // by concatenation
        String f = """
            Faculty of Computer
            and Information Sciences""";        // text block
        System.out.println(a + " | " + b + " | " + c);
        System.out.println(d + " | " + e);
        System.out.println(f);
    }
}
```

Five ways to get a String

codeoutput/Create.java — Java

```java
public class Create {
    public static void main(String[] args) {
        String a = "Yeditepe";                  // literal
        String b = new String("Yeditepe");      // new object
        char[] letters = {'J', 'a', 'v', 'a'};
        String c = new String(letters);         // from chars
        String d = String.valueOf(2026);        // from a number
        String e = "" + 3.14;  // by concatenation
        String f = """
            Faculty of Computer
            and Information Sciences""";        // text block
        System.out.println(a + " | " + b + " | " + c);
        System.out.println(d + " | " + e);
        System.out.println(f);
    }
}
```

Output
codeoutput/Create.txt

```
Yeditepe | Yeditepe | Java
2026 | 3.14
Faculty of Computer
and Information Sciences
```

length() and charAt(i)

codeoutput/LengthCharAt.java **Java**

```java
public class LengthCharAt {
    public static void main(String[] args) {
        String s = "Yeditepe";
        System.out.println("length: " + s.length());
        System.out.println("first : " + s.charAt(0));
        System.out.println("last  : " + s.charAt(s.length() - 1));
        for (int i = 0; i < s.length(); i++) {
            System.out.print(i + ":" + s.charAt(i) + " ");
        }
        System.out.println();
        char c = s.charAt(2);
        System.out.println(c + " has code " + (int) c);
    }
}
```

Valid indices are 0 to `length()-1`. A `char` is a number: `(int) c` shows its code.

length() and charAt(i)

codeoutput/LengthCharAt.java — Java

```java
public class LengthCharAt {
    public static void main(String[] args) {
        String s = "Yeditepe";
        System.out.println("length: " + s.length());
        System.out.println("first : " + s.charAt(0));
        System.out.println("last  : " + s.charAt(s.length() - 1));
        for (int i = 0; i < s.length(); i++) {
            System.out.print(i + ":" + s.charAt(i) + " ");
        }
        System.out.println();
        char c = s.charAt(2);
        System.out.println(c + " has code " + (int) c);
    }
}
```

Output codeoutput/LengthCharAt.txt

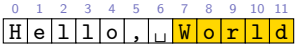
```
length: 8
first : Y
last  : e
0:Y 1:e 2:d 3:i 4:t 5:e 6:p 7:e
d has code 100
```

Valid indices are 0 to `length()-1`. A `char` is a number: `(int) c` shows its code.

substring(begin, end)

codeoutput/Substring.java **Java**

```java
public class Substring {
    public static void main(String[] args) {
        String s = "Hello, World";
        System.out.println(s.substring(7));         // to the end
        System.out.println(s.substring(0, 5));      // [0, 5)
        System.out.println(s.substring(7, 12));
        System.out.println(s.substring(3, 3).isEmpty());
        String file = "report.final.pdf";
        int dot = file.lastIndexOf('.');
        System.out.println(file.substring(0, dot));
        System.out.println(file.substring(dot + 1));
    }
}
```

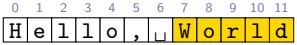


substring(7, 12): from index 7 **up to, not including** 12.

substring(begin, end)

codeoutput/Substring.java — Java

```java
public class Substring {
    public static void main(String[] args) {
        String s = "Hello, World";
        System.out.println(s.substring(7));         // to the end
        System.out.println(s.substring(0, 5));      // [0, 5)
        System.out.println(s.substring(7, 12));
        System.out.println(s.substring(3, 3).isEmpty());
        String file = "report.final.pdf";
        int dot = file.lastIndexOf('.');
        System.out.println(file.substring(0, dot));
        System.out.println(file.substring(dot + 1));
    }
}
```



substring(7, 12): from index 7 **up to, not including** 12.

Output
codeoutput/Substring.txt

```
World
Hello
World
true
report.final
pdf
```

Searching and Comparing

Searching: indexOf, contains, ...

codeoutput/Search.java — Java

```java
public class Search {
    public static void main(String[] args) {
        String s = "she sells sea shells";
        System.out.println(s.indexOf("se"));
        System.out.println(s.indexOf("se", 5));    // start at 5
        System.out.println(s.lastIndexOf('s'));
        System.out.println(s.indexOf("fish"));
        System.out.println(s.contains("sea"));
        System.out.println(s.startsWith("she"));
        System.out.println(s.endsWith("lls"));
        int count = 0;
        int i = s.indexOf('s');
        while (i >= 0) {                // every occurrence
            count++;
            i = s.indexOf('s', i + 1);
        }
        System.out.println("'s' occurs " + count + " times");
    }
}
```

`indexOf` returns `-1` when not found. The loop finds every occurrence by starting after the previous one.

Searching: indexOf, contains, ...

codeoutput/Search.java — Java

```java
public class Search {
    public static void main(String[] args) {
        String s = "she sells sea shells";
        System.out.println(s.indexOf("se"));
        System.out.println(s.indexOf("se", 5));     // start at 5
        System.out.println(s.lastIndexOf('s'));
        System.out.println(s.indexOf("fish"));
        System.out.println(s.contains("sea"));
        System.out.println(s.startsWith("she"));
        System.out.println(s.endsWith("lls"));
        int count = 0;
        int i = s.indexOf('s');
        while (i >= 0) {                // every occurrence
            count++;
            i = s.indexOf('s', i + 1);
        }
        System.out.println("'s' occurs " + count + " times");
    }
}
```

Output
codeoutput/Search.txt

```
4
10
19
-1
true
true
true
's' occurs 6 times
```

`indexOf` returns `-1` when not found. The loop finds every occurrence by starting after the previous one.

Comparing: equals and compareTo

codeoutput/Compare.java **Java**

```java
public class Compare {
    public static void main(String[] args) {
        String a = "java";
        String b = "Java";
        String c = new String("java");
        System.out.println(a == c);  // same object?
        System.out.println(a.equals(c));           // same text?
        System.out.println(a.equals(b));
        System.out.println(a.equalsIgnoreCase(b));
        System.out.println(a.compareTo(b));        // 'j' - 'J'
        System.out.println("apple".compareTo("banana"));
        System.out.println("app".compareTo("apple"));
        System.out.println(b.compareToIgnoreCase(a));
    }
}
```

== asks “same object?”; equals asks “same text?”. compareTo is negative, zero or positive.

Comparing: equals and compareTo

codeoutput/Compare.java — Java

```java
public class Compare {
    public static void main(String[] args) {
        String a = "java";
        String b = "Java";
        String c = new String("java");
        System.out.println(a == c);  // same object?
        System.out.println(a.equals(c));           // same text?
        System.out.println(a.equals(b));
        System.out.println(a.equalsIgnoreCase(b));
        System.out.println(a.compareTo(b));        // 'j' - 'J'
        System.out.println("apple".compareTo("banana"));
        System.out.println("app".compareTo("apple"));
        System.out.println(b.compareToIgnoreCase(a));
    }
}
```

Output
codeoutput/Compare.txt

```
false
true
false
true
32
-1
-2
0
```

== asks “same object?”; equals asks “same text?”. compareTo is negative, zero or positive.

Upper and lower case: the Turkish i

codeoutput/Case.java **Java**

```java
import java.util.Locale;

public class Case {
    public static void main(String[] args) {
        Locale tr = Locale.forLanguageTag("tr");
        String city = "istanbul";
        String word = "TITLE";
        System.out.println(city.toUpperCase(Locale.ROOT));
        System.out.println(city.toUpperCase(tr));
        System.out.println(word.toLowerCase(Locale.ROOT));
        System.out.println(word.toLowerCase(tr));
        System.out.println("ÇİĞDEM".toLowerCase(tr));
        System.out.println("ılık".toUpperCase(Locale.ROOT));
    }
}
```

Pitfall: `toUpperCase()` without a locale uses the computer's language. On a Turkish computer `"title".toUpperCase()` is `"TİTLE"`. Use `Locale.ROOT` for program text (keywords, file names).

Upper and lower case: the Turkish i

codeoutput/Case.java — Java

```java
import java.util.Locale;

public class Case {
    public static void main(String[] args) {
        Locale tr = Locale.forLanguageTag("tr");
        String city = "istanbul";
        String word = "TITLE";
        System.out.println(city.toUpperCase(Locale.ROOT));
        System.out.println(city.toUpperCase(tr));
        System.out.println(word.toLowerCase(Locale.ROOT));
        System.out.println(word.toLowerCase(tr));
        System.out.println("ÇİĞDEM".toLowerCase(tr));
        System.out.println("ılık".toUpperCase(Locale.ROOT));
    }
}
```

Output codeoutput/Case.txt

```
ISTANBUL
İSTANBUL
title
tıtle
çiğdem
ILIK
```

Pitfall: `toUpperCase()` without a locale uses the computer's language. On a Turkish computer `"title".toUpperCase()` is `"TİTLE"`. Use `Locale.ROOT` for program text (keywords, file names).

Transforming

Trimming, splitting and joining

codeoutput/TrimSplit.java **Java**

```java
import java.util.Arrays;

public class TrimSplit {
    public static void main(String[] args) {
        String raw = "   Ali , Ayşe,Can ,  Deniz  ";
        System.out.println("[" + raw.strip() + "]");
        String[] parts = raw.split(",");
        System.out.println(parts.length + " parts");
        System.out.println(Arrays.toString(parts));
        for (int i = 0; i < parts.length; i++) {
            parts[i] = parts[i].strip();
        }
        System.out.println(Arrays.toString(parts));
        System.out.println(String.join(" - ", parts));
        String line = "one   two\tthree";
        System.out.println(Arrays.toString(line.split("\\s+")));
        System.out.println("   ".isBlank() + " " + "   ".isEmpty());
    }
}
```

`split` takes a **regular expression**: `"\\s+"` means “one or more white-space characters.

Trimming, splitting and joining

codeoutput/TrimSplit.java — Java

```java
import java.util.Arrays;

public class TrimSplit {
    public static void main(String[] args) {
        String raw = "   Ali , Ayşe,Can ,  Deniz  ";
        System.out.println("[" + raw.strip() + "]");
        String[] parts = raw.split(",");
        System.out.println(parts.length + " parts");
        System.out.println(Arrays.toString(parts));
        for (int i = 0; i < parts.length; i++) {
            parts[i] = parts[i].strip();
        }
        System.out.println(Arrays.toString(parts));
        System.out.println(String.join(" - ", parts));
        String line = "one   two\tthree";
        System.out.println(Arrays.toString(line.split("\\s+")));
        System.out.println("   ".isBlank() + " " + "   ".isEmpty());
    }
}
```

Output
codeoutput/TrimSplit.txt

```
[Ali , Ayşe,Can ,  Deniz]
4 parts
[   Ali ,  Ayşe, Can ,   Deniz  ]
[Ali, Ayşe, Can, Deniz]
Ali - Ayşe - Can - Deniz
[one, two, three]
true false
```

`split` takes a **regular expression**: `"\\s+"` means “one or more white-space characters.

Replacing and repeating

codeoutput/Replace.java **Java**

```java
public class Replace {
    public static void main(String[] args) {
        String s = "banana";
        System.out.println(s.replace('a', 'o'));
        System.out.println(s.replace("an", "AN"));
        System.out.println(s.replaceFirst("an", "_"));
        String phone = "0 (212) 555-0123";
        // regex [^0-9]: any character that is not a digit
        System.out.println(phone.replaceAll("[^0-9]", ""));
        System.out.println("ab".repeat(3));
        System.out.println("-".repeat(20));
        System.out.println(s);  // unchanged!
    }
}
```

The last line proves it again: none of these methods changes `s`; each returns a new string.

Replacing and repeating

codeoutput/Replace.java — Java

```java
public class Replace {
    public static void main(String[] args) {
        String s = "banana";
        System.out.println(s.replace('a', 'o'));
        System.out.println(s.replace("an", "AN"));
        System.out.println(s.replaceFirst("an", "_"));
        String phone = "0 (212) 555-0123";
        // regex [^0-9]: any character that is not a digit
        System.out.println(phone.replaceAll("[^0-9]", ""));
        System.out.println("ab".repeat(3));
        System.out.println("-".repeat(20));
        System.out.println(s);  // unchanged!
    }
}
```

Output
codeoutput/Replace.txt

```
bonono
bANANa
b_ana
02125550123
ababab
--------------------
banana
```

The last line proves it again: none of these methods changes `s`; each returns a new string.

Formatting numbers and text

codeoutput/Format.java **Java**

```java
import java.util.Locale;

public class Format {
    public static void main(String[] args) {
        String name = "Ayşe";
        int n = 6;
        double gpa = 3.4567;
        String s = String.format("%s has %d credits", name, n);
        System.out.println(s);
        System.out.println(String.format("GPA: %.2f", gpa));
        Locale tr = Locale.forLanguageTag("tr");
        System.out.println(String.format(tr, "GPA: %.2f", gpa));
        System.out.printf("[%-8s][%8s]%n", name, name);
        System.out.printf("[%5d][%05d]%n", n, n);
        System.out.println("%s: %.1f%%".formatted("rate", 87.25));
    }
}
```

`%s` text, `%d` integer, `%.2f` two decimals, `%-8s` left-aligned in 8 columns, `%n` new line. With the Turkish locale the decimal separator is a comma.

Formatting numbers and text

codeoutput/Format.java — Java

```java
import java.util.Locale;

public class Format {
    public static void main(String[] args) {
        String name = "Ayşe";
        int n = 6;
        double gpa = 3.4567;
        String s = String.format("%s has %d credits", name, n);
        System.out.println(s);
        System.out.println(String.format("GPA: %.2f", gpa));
        Locale tr = Locale.forLanguageTag("tr");
        System.out.println(String.format(tr, "GPA: %.2f", gpa));
        System.out.printf("[%-8s][%8s]%n", name, name);
        System.out.printf("[%5d][%05d]%n", n, n);
        System.out.println("%s: %.1f%%".formatted("rate", 87.25));
    }
}
```

Output
codeoutput/Format.txt

```
Ayşe has 6 credits
GPA: 3.46
GPA: 3,46
[Ayşe    ][    Ayşe]
[    6][00006]
rate: 87.3%
```

`%s` text, `%d` integer, `%.2f` two decimals, `%-8s` left-aligned in 8 columns, `%n` new line. With the Turkish locale the decimal separator is a comma.

Strings and numbers

codeoutput/Convert.java — Java

```java
public class Convert {
    public static void main(String[] args) {
        int n = Integer.parseInt("42");
        double x = Double.parseDouble("2.5");
        System.out.println(n + 1);       // a number
        System.out.println("42" + 1);    // a string
        System.out.println(x * 2);
        String s = String.valueOf(n);    // int to String
        System.out.println(s.length());
        char[] cs = "hello".toCharArray();
        cs[0] = 'j';
        System.out.println(new String(cs));
        try {
            Integer.parseInt("4 2");
        } catch (NumberFormatException e) {
            System.out.println("error: " + e.getMessage());
        }
    }
}
```

`"42" + 1` is `"421"`: as soon as one side of `+` is a `String`, `+` means concatenation.

Strings and numbers

codeoutput/Convert.java — Java

```java
public class Convert {
    public static void main(String[] args) {
        int n = Integer.parseInt("42");
        double x = Double.parseDouble("2.5");
        System.out.println(n + 1);       // a number
        System.out.println("42" + 1);    // a string
        System.out.println(x * 2);
        String s = String.valueOf(n);    // int to String
        System.out.println(s.length());
        char[] cs = "hello".toCharArray();
        cs[0] = 'j';
        System.out.println(new String(cs));
        try {
            Integer.parseInt("4 2");
        } catch (NumberFormatException e) {
            System.out.println("error: " + e.getMessage());
        }
    }
}
```

Output
codeoutput/Convert.txt

```
43
421
5.0
2
jello
error: For input string:
     "4 2"
```

`"42" + 1` is `"421"`: as soon as one side of `+` is a `String`, `+` means concatenation.

Building Strings

StringBuilder: a mutable string

codeoutput/Builder.java — Java

```java
public class Builder {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Java");
        sb.append(' ').append(21).append('!');
        System.out.println(sb);
        sb.insert(0, "Hello ");
        System.out.println(sb);
        sb.setCharAt(0, 'h');
        sb.deleteCharAt(sb.length() - 1);
        System.out.println(sb + " (length " + sb.length() + ")");
        System.out.println(new StringBuilder("stressed").reverse());
        String result = sb.toString();  // immutable again
        System.out.println(result.toUpperCase());
    }
}
```

Methods return the builder itself, so calls can be chained: `sb.append(a).append(b)`.

StringBuilder: a mutable string

codeoutput/Builder.java — Java

```java
public class Builder {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Java");
        sb.append(' ').append(21).append('!');
        System.out.println(sb);
        sb.insert(0, "Hello ");
        System.out.println(sb);
        sb.setCharAt(0, 'h');
        sb.deleteCharAt(sb.length() - 1);
        System.out.println(sb + " (length " + sb.length() + ")");
        System.out.println(new StringBuilder("stressed").reverse());
        String result = sb.toString();  // immutable again
        System.out.println(result.toUpperCase());
    }
}
```

Output
codeoutput/Builder.txt

```
Java 21!
Hello Java 21!
hello Java 21 (length 13)
desserts
HELLO JAVA 21
```

Methods return the builder itself, so calls can be chained: `sb.append(a).append(b)`.

Word by word: title case

codeoutput/TitleCase.java **Java**

```java
public class TitleCase {
    public static void main(String[] args) {
        String s = "the quick brown fox";
        StringBuilder sb = new StringBuilder();
        for (String w : s.split(" ")) {
            if (sb.length() > 0) {
                sb.append(' ');
            }
            sb.append(Character.toUpperCase(w.charAt(0)));
            sb.append(w.substring(1));
        }
        System.out.println(sb);
        System.out.println(new StringBuilder(s).reverse());
        long os = s.chars().filter(c -> c == 'o').count();
        System.out.println(os + " o's");
    }
}
```

Word by word: title case

codeoutput/TitleCase.java — Java

```java
public class TitleCase {
    public static void main(String[] args) {
        String s = "the quick brown fox";
        StringBuilder sb = new StringBuilder();
        for (String w : s.split(" ")) {
            if (sb.length() > 0) {
                sb.append(' ');
            }
            sb.append(Character.toUpperCase(w.charAt(0)));
            sb.append(w.substring(1));
        }
        System.out.println(sb);
        System.out.println(new StringBuilder(s).reverse());
        long os = s.chars().filter(c -> c == 'o').count();
        System.out.println(os + " o's");
    }
}
```

Output
codeoutput/TitleCase.txt

```
The Quick Brown Fox
xof nworb kciuq eht
2 o's
```

Classifying characters: `Character`

codeoutput/CharClass.java **Java**

```java
public class CharClass {
    public static void main(String[] args) {
        String s = "Ders 101: Java & Strings!";
        int letters = 0, digits = 0, spaces = 0, others = 0;
        for (char c : s.toCharArray()) {
            if (Character.isLetter(c)) letters++;
            else if (Character.isDigit(c)) digits++;
            else if (Character.isWhitespace(c)) spaces++;
            else others++;
        }
        System.out.println("letters: " + letters);
        System.out.println("digits : " + digits);
        System.out.println("spaces : " + spaces);
        System.out.println("others : " + others);
        System.out.println(Character.toUpperCase('ş'));
        System.out.println(Character.isLetter('ğ'));
        System.out.println('7' - '0' + 1);
    }
}
```

`Character.isLetter` knows Turkish letters too. `'7' - '0'` turns a digit character into its value.

Classifying characters: `Character`

codeoutput/CharClass.java — Java

```java
public class CharClass {
    public static void main(String[] args) {
        String s = "Ders 101: Java & Strings!";
        int letters = 0, digits = 0, spaces = 0, others = 0;
        for (char c : s.toCharArray()) {
            if (Character.isLetter(c)) letters++;
            else if (Character.isDigit(c)) digits++;
            else if (Character.isWhitespace(c)) spaces++;
            else others++;
        }
        System.out.println("letters: " + letters);
        System.out.println("digits : " + digits);
        System.out.println("spaces : " + spaces);
        System.out.println("others : " + others);
        System.out.println(Character.toUpperCase('ş'));
        System.out.println(Character.isLetter('ğ'));
        System.out.println('7' - '0' + 1);
    }
}
```

Output
codeoutput/CharClass.txt

```
letters: 15
digits : 3
spaces : 4
others : 3
Ş
true
8
```

`Character.isLetter` knows Turkish letters too. `'7' - '0'` turns a digit character into its value.

Modern Java

Text blocks (Java 15+)

codeoutput/TextBlock.java **Java**

```java
public class TextBlock {
    public static void main(String[] args) {
        String json = """
            {
              "name": "Ayşe",
              "year": 1
            }
            """;
        System.out.print(json);
        System.out.println("lines: " + json.lines().count());
        String html = """
            <p>Hello, \
            World</p>""";
        System.out.println(html);
    }
}
```

A text block starts with """ and a new line. The common indentation is removed; a \ at the end of a line joins it with the next one.

Text blocks (Java 15+)

codeoutput/TextBlock.java **Java**

```java
public class TextBlock {
    public static void main(String[] args) {
        String json = """
            {
              "name": "Ayşe",
              "year": 1
            }
            """;
        System.out.print(json);
        System.out.println("lines: " + json.lines().count());
        String html = """
            <p>Hello, \
            World</p>""";
        System.out.println(html);
    }
}
```

Output
codeoutput/TextBlock.txt

```
{
  "name": "Ayşe",
  "year": 1
}
lines: 4
<p>Hello, World</p>
```

A text block starts with """ and a new line. The common indentation is removed; a \ at the end of a line joins it with the next one.

switch on strings

codeoutput/SwitchString.java **Java**

```java
import java.util.Locale;

public class SwitchString {
    static String kind(String day) {
        return switch (day.toLowerCase(Locale.ROOT)) {
            case "saturday", "sunday" -> "weekend";
            case "monday", "tuesday", "wednesday",
                 "thursday", "friday" -> "weekday";
            default -> "unknown: " + day;
        };
    }

    public static void main(String[] args) {
        String[] days = {"Monday", "SUNDAY", "FRIDAY", "Funday"};
        for (String d : days) {
            System.out.println(d + " -> " + kind(d));
        }
    }
}
```

A `switch` can test strings (with `equals`, never `==`). Normalize the case first, with `Locale.ROOT`.

switch on strings

codeoutput/SwitchString.java **Java**

```java
import java.util.Locale;

public class SwitchString {
    static String kind(String day) {
        return switch (day.toLowerCase(Locale.ROOT)) {
            case "saturday", "sunday" -> "weekend";
            case "monday", "tuesday", "wednesday",
                 "thursday", "friday" -> "weekday";
            default -> "unknown: " + day;
        };
    }

    public static void main(String[] args) {
        String[] days = {"Monday", "SUNDAY", "FRIDAY", "Funday"};
        for (String d : days) {
            System.out.println(d + " -> " + kind(d));
        }
    }
}
```

Output
codeoutput/SwitchString.txt

```
Monday -> weekday
SUNDAY -> weekend
FRIDAY -> weekday
Funday -> unknown: Funday
```

A `switch` can test strings (with `equals`, never `==`). Normalize the case first, with `Locale.ROOT`.

Summary

Cheat sheet

Method	Gives
`length()`	number of chars
`charAt(i)`	char at index `i`
`substring(b, e)`	chars `b..e-1`
`indexOf(x)`	first index or `-1`
`contains(x)`	`boolean`
`startsWith/endsWith`	`boolean`
`equals/equalsIgnoreCase`	same text?
`compareTo(x)`	< 0, 0, > 0

Method	Gives (a new string)
`toUpperCase(locale)`	upper case
`strip()`	no outer spaces
`split(regex)`	`String[]`
`String.join(sep, ...)`	joined string
`replace(a, b)`	all a replaced
`repeat(n)`	n copies
`String.format(...)`	formatted text
`Integer.parseInt(s)`	`int` (not a string)

Exercises

1. Print a name in reverse with a loop, then with `StringBuilder.reverse()`.
2. Count the vowels (`a e ı i o ö u ü`) in a Turkish sentence.
3. Given `"surname, name"`, print `"Name SURNAME"` (use `indexOf`, `substring`, `strip`).
4. Read a CSV line `"Ali,3.25,6"` and print it as a formatted table row with `String.format`.
5. Check whether a word is a palindrome, ignoring case (`"Kayak"`).
6. Turn `"hello world from java"` into `"helloWorldFromJava"` (camel case).
7. Explain the output of `System.out.println(1 + 2 + "3" + 4 + 5);` and then run it.

Questions?